



Roteiro de Atividade Prática

Nome: _____ Turma: _____

Título da atividade: Escalabilidade e performance em arquitetura de aplicações back-end

1. Introdução aos conceitos

Objetivo: apresentar os conceitos de escalabilidade e performance, explorando como eles são tratados em arquiteturas monolíticas e de microsserviços.

- **Escalabilidade:**

- **Definição:** a capacidade de um sistema crescer em capacidade para lidar com um aumento na carga de trabalho. Isso pode ser feito horizontalmente (adicionando mais máquinas/servidores) ou verticalmente (aumentando a capacidade de uma única máquina).
- **Escalabilidade em arquitetura monolítica:** geralmente mais difícil de escalar horizontalmente, pois a aplicação inteira precisa ser replicada. Escalabilidade vertical é limitada e pode se tornar cara.
- **Escalabilidade em microsserviços:** facilita a escalabilidade horizontal, pois cada serviço pode ser escalado independentemente com base em sua carga específica.

- **Performance:**

- **Definição:** o quão rápido e eficiente um sistema responde às requisições. A performance pode ser impactada por vários fatores, como tempo de resposta, tempo de processamento e eficiência de uso de recursos.



- **Performance em arquitetura monolítica:** pode ser eficiente em sistemas pequenos, mas à medida que o sistema cresce, as interdependências podem causar gargalos de performance.
- **Performance em microsserviços:** pode melhorar a performance geral, permitindo otimizações específicas por serviço, mas também pode introduzir latência devido à comunicação entre serviços.

2. Explorando a escalabilidade e performance em diferentes arquiteturas

Objetivo: discutir como as arquiteturas monolítica e de microsserviços abordam a escalabilidade e a performance, com exemplos práticos e teóricos.

- **Escalabilidade em arquitetura monolítica:**
 - **Exemplo conceitual:** um sistema de *e-commerce* monolítico que precisa escalar para atender ao aumento de usuários durante a Black Friday. A escalabilidade será limitada pela capacidade do servidor principal, e qualquer falha afetará todo o sistema.
 - **Discussão:** Como você poderia escalar esse sistema? Quais os desafios?
- **Escalabilidade em microsserviços:**
 - **Exemplo conceitual:** o mesmo sistema de *e-commerce*, mas agora cada funcionalidade (usuários, produtos, pedidos) é um serviço separado. Durante a Black Friday, apenas os serviços de pedidos e pagamentos precisam ser escalados, otimizando recursos.
 - **Discussão:** Como essa abordagem facilita a escalabilidade? Quais são os novos desafios introduzidos?
- **Performance em arquitetura monolítica:**
 - **Exemplo conceitual:** no sistema monolítico, se o módulo de pagamento estiver lento, ele afeta o desempenho de todo o sistema.
 - **Discussão:** Como você pode otimizar a performance em um sistema monolítico? Quais são as limitações?
- **Performance em microsserviços:**



- **Exemplo conceitual:** no sistema de microsserviços, o serviço de pagamento pode ser otimizado e escalado independentemente para melhorar a performance sem impactar outros serviços.
- **Discussão:** Como a performance pode ser otimizada em uma arquitetura de microsserviços? Quais são as armadilhas comuns?

3. Preparação para a atividade prática

Objetivo: preparar os participantes para a implementação prática, explicando o que será construído e como cada parte do código se relaciona com os conceitos aprendidos.

- **Cenário da atividade:**
 - Os participantes vão implementar um sistema simples em Python que simula uma loja on-line. Primeiro, eles vão construir uma versão monolítica, em seguida, refatorar para microsserviços, focando em escalabilidade e performance.
- **Explicação dos passos:**
 - **Passo 1:** implementar uma versão monolítica da loja on-line, com todas as funcionalidades integradas;
 - **Passo 2:** refatorar o código para separar as funcionalidades em microsserviços, aplicando técnicas para melhorar a escalabilidade e performance;
 - **Passo 3:** testar a aplicação para analisar as melhorias de performance e escalabilidade.

4. Atividade prática em Python

Objetivo: colocar em prática os conceitos e padrões aprendidos, implementando uma loja on-line em duas etapas: primeiro como uma arquitetura monolítica e depois refatorando para microsserviços.



Passo 1: Implementação monolítica

```
class OnlineStore:
    def __init__(self):
        self.users = []
        self.products = []
        self.orders = []

    def register_user(self, user):
        self.users.append(user)

    def add_product(self, product):
        self.products.append(product)

    def place_order(self, user, product):
        if product in self.products:
            self.orders.append((user, product))
            return "Order placed successfully"
        return "Product not available"

# Exemplo de uso
store = OnlineStore()
store.register_user("Alice")
store.add_product("Laptop")
print(store.place_order("Alice", "Laptop"))
```

Passo 2: Refatoração para microsserviços

```
class UserService:
    def __init__(self):
        self.users = []

    def register_user(self, user):
        self.users.append(user)
```



```
class ProductService:
    def __init__(self):
        self.products = []

    def add_product(self, product):
        self.products.append(product)

    def check_availability(self, product):
        return product in self.products

class OrderService:
    def __init__(self):
        self.orders = []

    def place_order(self, user, product, product_service):
        if product_service.check_availability(product):
            self.orders.append((user, product))
            return "Order placed successfully"
        return "Product not available"

# Exemplo de uso com microsserviços
user_service = UserService()
product_service = ProductService()
order_service = OrderService()

user_service.register_user("Alice")
product_service.add_product("Laptop")
print(order_service.place_order("Alice", "Laptop", product_service))
```

Passo 3: Testes e análise

- **Testes:** execute as duas versões, monolítica e microsserviços, e compare como cada uma lida com um aumento na carga (simulado por loops ou múltiplas chamadas).



- **Análise:** discuta como a refatoração para microsserviços impactou a escalabilidade e performance. Quais melhorias foram observadas? Houve algum novo desafio?

5. Discussão e reflexão

Objetivo: refletir sobre o que foi aprendido e discutir como a escalabilidade e performance são afetadas pela arquitetura escolhida.

- **Perguntas para reflexão:**
 - Como a refatoração para microsserviços afetou a escalabilidade do sistema?
 - Quais foram as principais diferenças de performance entre as duas arquiteturas?
 - Que outros aspectos poderiam ser otimizados para melhorar ainda mais a performance e escalabilidade?

1: Permite a escalabilidade horizontal independente, facilitando que cada serviço cresça conforme sua carga específica

2: A arquitetura monolítica gera gargalos ao crescer , enquanto microsserviços permitem otimizações focadas em cada serviço, apesar da latência de comunicação entre eles.

3: É possível realizar otimizações isoladas em serviços lentos e aplicar melhorias de hardware vertical ou horizontal
